

Defining the Entry Model

For a user to record what they’ve been learning about chess and rock climbing, we need to define a model for the kinds of entries users can make in their learning logs. Each entry needs to be associated with a particular topic. This relationship is called a *many-to-one relationship*, meaning many entries can be associated with one topic.

Here’s the code for the Entry model. Place it in your *models.py* file:

```
models.py  from django.db import models

class Topic(models.Model):
    --snip--

❶ class Entry(models.Model):
    """Something specific learned about a topic."""
    ❷ topic = models.ForeignKey(Topic, on_delete=models.CASCADE)
    ❸ text = models.TextField()
    date_added = models.DateTimeField(auto_now_add=True)

    ❹ class Meta:
        verbose_name_plural = 'entries'

    def __str__(self):
        """Return a simple string representing the entry."""
    ❺ return f"{self.text[:50]}..."
```

The Entry class inherits from Django’s base Model class, just as Topic did ❶. The first attribute, topic, is a ForeignKey instance ❷. A *foreign key* is a database term; it’s a reference to another record in the database. This is the code that connects each entry to a specific topic. Each topic is assigned a *key*, or ID, when it’s created. When Django needs to establish a connection between two pieces of data, it uses the keys associated with each piece of information. We’ll use these connections shortly to retrieve all the entries associated with a certain topic. The `on_delete=models.CASCADE` argument tells Django that when a topic is deleted, all the entries associated with that topic should be deleted as well. This is known as a *cascading delete*.

Next is an attribute called text, which is an instance of TextField ❸. This kind of field doesn’t need a size limit, because we don’t want to limit the size of individual entries. The date_added attribute allows us to present entries in the order they were created, and to place a timestamp next to each entry.

The Meta class is nested inside the Entry class ❹. The Meta class holds extra information for managing a model; here, it lets us set a special attribute telling Django to use Entries when it needs to refer to more than one entry. Without this, Django would refer to multiple entries as Entry.

The `__str__()` method tells Django which information to show when it refers to individual entries. Because an entry can be a long body of text, `__str__()` returns just the first 50 characters of text ❺. We also add an ellipsis to clarify that we’re not always displaying the entire entry.